

- [Parcours de Formation](#)

- 1. DevSecOps



- Parcours d'introduction



- Sécurité et DevSecOps



- Pratiques DevSecOps



- Équipes et métiers



- DevSecOps



- Notions Importantes



- 2. Administrer des Systèmes



- 3. Les réseaux informatiques



- 4. Développer des applications



- 5. Sécuriser l'infrastructure



- 6. Documenter



- 7. Administrer des services



- 8. Virtualiser des ressources



- 9. Maîtriser l'Infra-as-code



- [Introduction](#)



- Gestion de Configurations



- [Introduction](#)



- Ansible



- [Introduction](#)



- La Formation

- Complète



- Les Principaux

- Modules



- Les Meilleurs

- Pratiques



- [Ecrire du](#)

- [code ansible](#)

- [correctement](#)



Sur cette page >

Home > docs > infra-as-code > gestion-de-configuration > ansible > ansible-best-practices-tips

Les meilleures pratiques d'Ansible

Mise à jour : 18/06/2025



ANSIBLE

Ça fait maintenant 7 ans que j'utilise Ansible quotidiennement pour provisionner et configurer des infrastructures. Comme j'ai vu plusieurs implémentations au fil de mes changements de poste, je vous propose deux billets sur les bonnes pratiques et les meilleurs patterns Ansible que j'ai pu rencontrer.

Astuce

Si vous êtes débutant sur **Ansible**, je vous propose de commencer par [ce billet](#). En effet, avant de lire celui-ci, il faut maîtriser le vocabulaire **Ansible** pour le comprendre.

Qualité de code

Le DSL Ansible : YAML

Ansible utilise le YAML comme DSL. Un fichier yaml débute toujours avec --- sur la première ligne et ... sur la dernière ligne.

Utilisez deux espaces pour l'indentation.

```
---
- name: update web servers
  hosts: webservers
  become: true

  tasks:
    - name: ensure apache is at the latest version
      ansible.builtin.package:
        name: httpd
        state: latest
...

```

Utilisez des espaces cohérents : Pour bien séparer les choses et améliorer la lisibilité, envisagez de laisser une ligne vierge entre les blocks, les tasks, les handlers.

Outils d'assistance d'écriture

Pour vous aider à écrire votre code **Ansible**, utilisez un éditeur permettant d'installer [l'extension fournie par Redhat](#).

Pour valider votre code, utilisez les outils [ansible-lint](#) ou [spotter](#) qui vous indiquera comment en améliorer la qualité.

Mettre en place des règles de nommage

Utilisez une stratégie de nommage cohérente pour les variables, les playbooks, les tâches, les rôles et modules.

Exemples de règles de nommage :

- **Tous les fichiers doivent utiliser l'extension .yml et non .YML, ni .yaml, ...**
- **Toutes les variables devant être passé dans la ligne de commande débute par cli_.**
- **Toutes les variables globales débutent par g_.**
- **Toutes les variables utilisées la convention snake case.**
- ****Toutes les variables globales des rôles doivent posséder le nom du rôle comme préfixe**
install_nginx_g_nginx_version.
- ****Seules les variables pouvant être modifié doivent être placé dans default/main.yml. Toutes les autres dans vars/main.yml.**
- **Toutes les variables se trouvant dans vars/main.yml doivent commencer par un _.**
- **On utilise _ comme séparateur de noms de variables, de nom de playbooks, de rôles ...**
- ****Les noms de variables Jinja sont encadrés par des espaces {{ variable_1 }}.**
- ...

Exemple de nommage de variables :

```
myappIP: "127.0.0.1" # incorrect
myapp_ip: "127.0.0.1" # incorrect
myapp_ip: "127.0.0.1" # correct

```

Mettre en place des règles de programmation

Sur le code

- [Automatiser vos Tests avec Molecule et Tox](#)
- [Les Meilleures Pratiques d'Ansible](#)
- [Optimiser les temps d'exécution d'Ansible](#)
- [Améliorer votre code Ansible avec Spotter](#)
- [L'extension VSCode Ansible](#)
- [Développer des Modules Ansible](#)
- [Développer et Installer des collections](#)
- [Les environnements d'exécution Ansible](#)
- [Ansible Navigator](#)
- [Ansible Pull](#)
- [Chef](#)
- [Puppet](#)
- [Rudder](#)
- [Provisionnement](#)
- [Eviter la dette technique](#)
- 10. Maitriser la conteneurisation
- 11. Mettre en place l'Observabilité
- 12. Cloud et DevOps
- 13. Ecrire des Pipelines CI-CD
- 14. Mon Homelab DevOps
- 15. MLOps
- Outils Indispensables
- [Glossaire Devops](#)
- [Mes favoris ♥](#)
- [Soutenez mon projet](#)
- [Mon Discord](#)
- [A Mon Propos](#)
- [Mentions Légales](#)
- [Rappel à la loi](#)

- Comme pour tout langage de programmation, **lorsque votre code nécessite une explication supplémentaire pour sa compréhension, n'hésitez pas à ajouter un commentaire**. Un commentaire débute par le caractère #.

```
...
- name: update web servers
  hosts: webservers # n'installe le package http que sur les serveurs webs
  become: true
...
```

- **Éviter de construire des usines à gaz !**: Comme tout langage de programmation, **Ansible** offre de nombreuses façons de découper son code. Une règle que je me fixe depuis toujours : **programmer en utilisant les algorithmes les plus simples possibles -> Votre code sera plus facile à maintenir**.
- **Dont Repeat Yourself** : Factoriser un maximum votre code en ayant recours aux rôles (un rôle rempli un objectif) et collections **Ansible**. **Les rôles doivent être stockés dans un gestionnaire de code et posséder un pipeline d'intégration** permettant de tester si le code fonctionne toujours à chaque montée de versions des dépendances : ansible-core, les collections, les librairies python, ...
- Pour les tests d'intégration la meilleure solution est d'utilisée [molecule](#) via les drivers mis à disposition : docker, podman, vagrant, ec2, gcp ...
- **Le recours aux modules raw, shell et command ne doit être fait qu'en dernier recours. En effet, ses modules cassent l'idempotence de votre code Ansible**.
- Découper vos fichiers yaml pour les rendre lisibles. Utilisez un fichier chapeau main.yml pour orchestrer d'autres taches de niveau inférieur en les regroupant par tâches.
- **Définissez toujours des valeurs par défaut pour vos variables**

```
"{{ some_variable | default('default_value') }}"
```

- **Indiquez toujours l'état des paramètres booléens** même si cela n'est pas nécessaire en raison de la valeur par défaut. **Rien ne dit qu'à la prochaine version la valeur par défaut change !**

```
- name: Creation d'un user applicatif
  ansible.builtin.user:
    name: "{{ user_appli_name }}"
    uid: "{{ user_appli_uid }}"
    group: "{{ user_appli_group_name }}"
    create_home: true
    shell: "/sbin/nologin"
    force: false
    append: false
    remove: false
    system: false
    move_home: false
    non_unique: false
    become: true
```

- Évitez un maximum le recours à ignore_errors. Pour cela, vous pouvez :
 - Pour toutes les taches qui ne doivent s'exécuter que si une des précédentes a provoqué un changement, pensez à utiliser les handlers.

```
tasks:
  - name: Création d'une base toto
    community.postgresql.postgresql_db:
      name: acme
      state: present
    notify: Execute script

handlers:
  - name: Execute script
    community.postgresql.postgresql_script:
      db: acme
      path: /var/lib/pgsql/insert.sql
      encoding: UTF-8
```

- Utilisez les conditions when :

```
tasks:
  - name: Création d'une base toto
    community.postgresql.postgresql_db:
      name: acme
      state: present
    register: r_create_database

  - name: Execute script
    when: r_create_database | changed
    community.postgresql.postgresql_script:
      db: acme
      path: /var/lib/pgsql/insert.sql
      encoding: UTF-8
...
```

Sur les inventaires

- **Utilisez des inventaires séparés** pour chaque environnement pour les isoler les uns des

- autres et éviter ainsi les erreurs en ciblant les mauvais environnements.
- Pensez à utiliser le regroupement dynamique au moment de l'exécution à l'aide du module `group_by`.
- Utilisez des groupes d'inventaire : Hôtes de groupe basés sur des attributs communs qu'ils pourraient partager (géographie, objectif, rôles, environnement). Attention à la [précédence des variables Ansible](#).
- Ne mettez dans l'inventaire que des variables globales propres à l'infrastructure. Pour les autres variables, mettez-les au niveau du playbook;
- Utilisez la commande `ansible-inventory --list` pour vérifier le résultat.

Sécurité

Gestion des secrets

On utilise tout git pour stocker nos données (playbooks, inventaires, rôles, collections ...) **Ansible**. Mais **attention à ne pas divulguer vos données secrètes** comme des **mots de passe**, des **clés SSH**, des **tokens d'API**, ...

Ansible met à disposition [ansible-vault](#) (un coffre-fort) qui permet de chiffrer ces données. Au besoin, ansible saura déchiffrer les données.

Utilisation des rôles et collections disponibles sur Ansible Galaxy

Faites preuve de diligence raisonnable lorsque vous utilisez des rôles disponibles depuis Ansible Galaxy : Validez leur contenu et choisissez des rôles auprès de contributeurs fiables : gueerlinguy, robert-de-bock

Stockez vos rôles dans vos référentiels de code et utilisez un pipeline pour en contrôler le contenu (pas de secrets).

Au moment d'exécuter un playbook sur la production

Si vous n'êtes pas l'auteur d'un playbook, vous pouvez :

- Utiliser l'option `--check` qui ne réalise aucune modification.
- Demander à juste visualiser les tâches qui s'exécuteront. Il faut juste ajouter l'option `--list-tasks` à la commande `ansible-playbook`.

```

ansible-playbook configure.yml --list-tasks

playbook: configure.yml

  play #1 (all:!bastion): all:!bastion  TAGS: []
    tasks:
      vérification que les machines sont accessibles  TAGS: []

  play #2 TAGS: []
    tasks:
      application/installation : Creation du groupe applicatif general  TAGS: []
      application/installation : Creation du user applicatif general  TAGS: []
      application/installation : Creation du répertoire data  TAGS: []

```

- Vous pouvez aussi valider pour chaque tâche quelles cibles sont concernées avec l'option `--list-hosts`.

```

ansible-playbook configure.yml --list-tasks --list-hosts
playbook: configure.yml

  play #1 (all:!bastion): all:!bastion  TAGS: []
    pattern: ['all:!bastion']
    hosts (4):
      machine1
      machine2
    tasks:
      vérification que les machines sont accessibles  TAGS: []

  play #2 (!rnvp:outils): !rnvp:outils  TAGS: []
    pattern: ['!rnvp:outils']
    hosts (3):
      bastion
      machine1
      machine2
    tasks:
      application/installation : Creation du groupe applicatif general  TAGS: []
      application/installation : Creation du user applicatif general  TAGS: []
      application/installation : Creation du répertoire data  TAGS: []

```

Puis loin

Si vous avez d'autres tips n'hésitez pas à me les remonter.

Partagez cet article :

[LinkedIn](#)[Reddit](#)[X](#)[Bluesky](#)[Facebook](#)



[Précédent](#)
[Automatiser](#)
[vos Tests](#)
[avec](#)
[Molecule et](#)
[Tox](#) →



[Suivant](#)
[Optimiser](#)
[les temps](#)
[d'exécution](#)
[d'Ansible](#)

À propos de l'auteur

Je suis **Stéphane Robert**, passionné par le [DevSecOps](#), l'infrastructure cloud et l'automatisation. J'écris sur ces sujets pour partager des connaissances et aider chacun à progresser dans ce domaine.

Ce site est **gratuit et accessible à tous**, avec des ressources sur [Terraform](#), [Ansible](#), [Kubernetes](#) et bien plus. Mon objectif est de proposer du contenu pratique et des guides détaillés pour faciliter l'apprentissage.

Vous aimez mes contenus ? Vous pouvez [me soutenir en m'offrant un café ☕](#). Cela m'aide à maintenir et enrichir ces ressources accessibles à tous.

Pour suivre mes publications, retrouvez-moi sur [LinkedIn](#) et [mon compte Github](#).